1 Pseudocode

In our submission, we prove two claims by employing an exhaustive search of matrices in $\mathrm{PSL}(2, \mathcal{O}_d)$ with a specified norm property. The pseudocode below details the procedure.

Input: $d \in \{1, 2, 3, 7, 11\}$

  `matrices` $\leftarrow$ empty list

  $S \leftarrow \left\{x \in \mathcal{O}_d : N(x) < \frac{1}{1-\kappa(d)}\right\}$; ▷ set of matrix entries

  for ordered tuples $(\alpha, \beta, \gamma, \delta)$, with $\alpha, \beta, \gamma, \delta \in S$, $\gamma \neq 0$, and $\alpha\delta - \beta\gamma = 1$ **do**

    $\left(M, \|M\|\right) \leftarrow \left(\left(\begin{smallmatrix} \alpha & \beta \\ \gamma & \delta \end{smallmatrix}\right), \max\{|\alpha|^2, |\beta|^2, |\gamma|^2, |\delta|^2\}\right)$; ▷ matrix and its norm

    $\theta \leftarrow$ Quotient$(-\delta, \gamma)$; ▷ output quotient $-\theta \in \mathcal{O}_d$

    $(\alpha, \beta) \leftarrow (\theta \cdot \alpha + \beta, -\alpha)$;

    $(\gamma, \delta) \leftarrow (\theta \cdot \gamma + \delta, -\gamma)$;

    $\left(N, \|N\|\right) \leftarrow \left(\left(\begin{smallmatrix} \alpha & \beta \\ \gamma & \delta \end{smallmatrix}\right), \max\{|\alpha|^2, |\beta|^2, |\gamma|^2, |\delta|^2\}\right)$; ▷ update matrix and its norm

    if $\|M\| < \|N\|$ **then**

      add (M, N) to `matrices`

    end if

  end for

  if `matrices` is an empty list **then**

    return "Claim holds for matrices in $\mathrm{PSL}(2, \mathcal{O}_d)$"

  else

    return "Claim does not hold for the pairs `matrices`"

  end if

Algorithm 1: Search and verification of claims in $\mathrm{PSL}(2, \mathcal{O}_d)$.

2 SageMath code for $\mathrm{PSL}(2, \mathbb{Z}[\mathrm{i}])$

Here we comment on the implementation for the search for matrices in $\mathrm{PSL}(2, \mathbb{Z}[\mathrm{i}])$. *Mutatis Mutandis,* the procedures for searching the other Euclidean Bianchi groups are similar.

We define the Gaussian field $\mathbb{Q}[\mathrm{i}]$ and fix the embedding that maps the generator ω, a solution of $\omega^2 + 1 = 0$, to $\omega \mapsto \mathrm{i}$. We also specify the Gaussian integers $\mathbb{Z}[\mathrm{i}]$.

```
sage: # CC.0 is the imaginary unit 'i'
      K.<w> = QuadraticField(-1, embedding=CC.0)
      OK = K.ring_of_integers()
```

SageMath does not automatically recognise rings, such as the integer rings $\mathcal{O}_d$, that are Euclidean domains. Thus for integers $a, b \in \mathcal{O}_d$, the function `a.quo_rem(b)[0]` will output the quotient a/b obtained by division in the field $\mathbb{Q}(\sqrt{-d})$. We circumvent this problem by implementing a search for the closest algebraic integer in $\mathcal{O}_d$ to the quotient a/b. As an aside, simply rounding the coordinates of an algebraic number does not work for finding the closest algebraic integer in an oblique lattice (e.g., the closest integer in $\mathcal{O}_7$).

```
sage: # Function that returns the closest algebraic integer in OK
      def closest_integer(field_quo):
```

```
        K = field_quo.parent()

        # {1, omega} is a basis for the integer lattice
        basis = OK.basis()

        # Embed the integer lattice and field_quo into RR^2. field_quo maps to
            ↪ field_target
        def embed(element):
            v = CC(element)
            return vector(RR, [v.real(), v.imag()])
        field_target = embed(field_quo)

        # Basis matrix B for the integer lattice
        B = matrix(RR, [embed(b) for b in basis]).transpose()

        # Exact coordinates of the field_target in the lattice basis
        coords = B.solve_right(field_target)

        # Variables that record the current closest integer
        best_norm = infinity
        best_integer = None

        # Search local neighbours for the closest integer to field_target
        import itertools
        options = [[floor(v)-1, floor(v), floor(v)+1] for v in coords]
        for p, q in itertools.product(*options):
            candidate = OK([p, q])
            # Norm of diff between candidate and field_target
            diff = embed(candidate) - field_target
            norm_diff = diff.dot_product(diff)

            # Update current closest integer
            if norm_diff < best_norm:
                best_norm = norm_diff
                best_integer = candidate

    return best_integer
```

Now we generate all 2×2 matrices with entries in $\mathbb{Z}[\mathrm{i}]$ and determinant 1. We also produce the list of updated matrices, where we employ the `closest_integer()` function defined above.

Recall that a matrix M and its update N in $\mathrm{PSL}(2, \mathcal{O}_d)$ are defined as follows:

$$M = \begin{pmatrix} \alpha & \beta \\ \gamma & \delta \end{pmatrix} \quad \text{and} \quad N = \begin{pmatrix} \theta\alpha+\beta & -\alpha \\ \theta\gamma+\delta & -\gamma \end{pmatrix}.$$

```
sage: # Matrix entries in ZZ[i]
      S = [K(0), K(1), w, w*w, w*w*w]
      T = [K(1), w, w*w, w*w*w]

      # Generate lists of matrices and their updates
      mats = []
      matsupdate = []

      for a in S:
        for b in S:
            for c in T:
                for d in S:
                    M = matrix(K, [[a, b], [c, d]])
                        if M.det() == 1:
```

```
                                # closest integer to quotient
                                theta = closest_integer(K(K(-d).quo_rem(K(c))[0]))
                                # update entry a
                                alpha = K(theta)*K(a) + K(b)
                                # update entry c
                                gamma = K(theta)*K(c) + K(d)
                                N = matrix(K, [[alpha, -a], [gamma, -c]])
                                mats.append(M)
                                matsupdate.append(N)
```

We compute the norms for the matrices and their updates.

```
sage: # Return the max of the normed elements in a matrix
      def max_abs_squared_norm(M):
        return max((x*x.galois_conjugate() for x in M.list()))

      # Generate lists for the norms of the matrices and their updates
      normmats = []
      normmatsupdate = []

      for m in mats:
        normmats.append(max_abs_squared_norm(m))

      for m in matsupdate:
        normmatsupdate.append(max_abs_squared_norm(m))
```

Output the list of matrices, updates, and their respective norms.

```
sage: result = list(zip(mats, matsupdate, normmats, normmatsupdate))

      # print the matrices, their updates, and the respective norms
      for i, (M, N, x, y) in enumerate(result, 1):
        print(f"#{i:3d} matrix {list(map(list, M.rows()))} update {list(map(list,
            ↪ N.rows()))} | norms=({x}, {y})")
```

Listing 1. Output candidate matrices, update matrices, and their respective norms. The claim for $d = 1$ is easily verified from the output norms.

```
  1. matrix [[0, 1], [-1, 0]] update [[1, 0], [0, 1]] | norms=(1, 1)
  2. matrix [[0, 1], [-1, 1]] update [[1, 0], [0, 1]] | norms=(1, 1)
  3. matrix [[0, 1], [-1, w]] update [[1, 0], [0, 1]] | norms=(1, 1)
  4. matrix [[0, 1], [-1, -1]] update [[1, 0], [0, 1]] | norms=(1, 1)
  5. matrix [[0, 1], [-1, -w]] update [[1, 0], [0, 1]] | norms=(1, 1)
  6. matrix [[0, w], [w, 0]] update [[w, 0], [0, -w]] | norms=(1, 1)
  7. matrix [[0, w], [w, 1]] update [[w, 0], [0, -w]] | norms=(1, 1)
  8. matrix [[0, w], [w, w]] update [[w, 0], [0, -w]] | norms=(1, 1)
  9. matrix [[0, w], [w, -1]] update [[w, 0], [0, -w]] | norms=(1, 1)
 10. matrix [[0, w], [w, -w]] update [[w, 0], [0, -w]] | norms=(1, 1)
 11. matrix [[0, -1], [1, 0]] update [[-1, 0], [0, -1]] | norms=(1, 1)
 12. matrix [[0, -1], [1, 1]] update [[-1, 0], [0, -1]] | norms=(1, 1)
 13. matrix [[0, -1], [1, w]] update [[-1, 0], [0, -1]] | norms=(1, 1)
 14. matrix [[0, -1], [1, -1]] update [[-1, 0], [0, -1]] | norms=(1, 1)
 15. matrix [[0, -1], [1, -w]] update [[-1, 0], [0, -1]] | norms=(1, 1)
 16. matrix [[0, -w], [-w, 0]] update [[-w, 0], [0, w]] | norms=(1, 1)
 17. matrix [[0, -w], [-w, 1]] update [[-w, 0], [0, w]] | norms=(1, 1)
 18. matrix [[0, -w], [-w, w]] update [[-w, 0], [0, w]] | norms=(1, 1)
 19. matrix [[0, -w], [-w, -1]] update [[-w, 0], [0, w]] | norms=(1, 1)
 20. matrix [[0, -w], [-w, -w]] update [[-w, 0], [0, w]] | norms=(1, 1)
```

```
21. matrix [[1, 0], [1, 1]] update [[-1, -1], [0, -1]] | norms=(1, 1)
22. matrix [[1, 0], [w, 1]] update [[w, -1], [0, -w]] | norms=(1, 1)
23. matrix [[1, 0], [-1, 1]] update [[1, -1], [0, 1]] | norms=(1, 1)
24. matrix [[1, 0], [-w, 1]] update [[-w, -1], [0, w]] | norms=(1, 1)
25. matrix [[1, 1], [-1, 0]] update [[1, -1], [0, 1]] | norms=(1, 1)
26. matrix [[1, w], [w, 0]] update [[w, -1], [0, -w]] | norms=(1, 1)
27. matrix [[1, -1], [1, 0]] update [[-1, -1], [0, -1]] | norms=(1, 1)
28. matrix [[1, -w], [-w, 0]] update [[-w, -1], [0, w]] | norms=(1, 1)
29. matrix [[w, 0], [1, -w]] update [[-1, -w], [0, -1]] | norms=(1, 1)
30. matrix [[w, 0], [w, -w]] update [[w, -w], [0, -w]] | norms=(1, 1)
31. matrix [[w, 0], [-1, -w]] update [[1, -w], [0, 1]] | norms=(1, 1)
32. matrix [[w, 0], [-w, -w]] update [[-w, -w], [0, w]] | norms=(1, 1)
33. matrix [[w, 1], [-1, 0]] update [[1, -w], [0, 1]] | norms=(1, 1)
34. matrix [[w, w], [w, 0]] update [[w, -w], [0, -w]] | norms=(1, 1)
35. matrix [[w, -1], [1, 0]] update [[-1, -w], [0, -1]] | norms=(1, 1)
36. matrix [[w, -w], [-w, 0]] update [[-w, -w], [0, w]] | norms=(1, 1)
37. matrix [[-1, 0], [1, -1]] update [[-1, 1], [0, -1]] | norms=(1, 1)
38. matrix [[-1, 0], [w, -1]] update [[w, 1], [0, -w]] | norms=(1, 1)
39. matrix [[-1, 0], [-1, -1]] update [[1, 1], [0, 1]] | norms=(1, 1)
40. matrix [[-1, 0], [-w, -1]] update [[-w, 1], [0, w]] | norms=(1, 1)
41. matrix [[-1, 1], [-1, 0]] update [[1, 1], [0, 1]] | norms=(1, 1)
42. matrix [[-1, w], [w, 0]] update [[w, 1], [0, -w]] | norms=(1, 1)
43. matrix [[-1, -1], [1, 0]] update [[-1, 1], [0, -1]] | norms=(1, 1)
44. matrix [[-1, -w], [-w, 0]] update [[-w, 1], [0, w]] | norms=(1, 1)
45. matrix [[-w, 0], [1, w]] update [[-1, w], [0, -1]] | norms=(1, 1)
46. matrix [[-w, 0], [w, w]] update [[w, w], [0, -w]] | norms=(1, 1)
47. matrix [[-w, 0], [-1, w]] update [[1, w], [0, 1]] | norms=(1, 1)
48. matrix [[-w, 0], [-w, w]] update [[-w, w], [0, w]] | norms=(1, 1)
49. matrix [[-w, 1], [-1, 0]] update [[1, w], [0, 1]] | norms=(1, 1)
50. matrix [[-w, w], [w, 0]] update [[w, w], [0, -w]] | norms=(1, 1)
51. matrix [[-w, -1], [1, 0]] update [[-1, w], [0, -1]] | norms=(1, 1)
52. matrix [[-w, -w], [-w, 0]] update [[-w, w], [0, w]] | norms=(1, 1)
```